

# Knowledge Graphs

Lydia Brorsson and Santiago Delgado

Semantic Data Management  
Master's degree in Data Science  
Facultat d'Informàtica de Barcelona  
Universitat Politècnica de Catalunya (UPC)

May 12, 2026

## A Knowledge Graph Modeling

The proposed model represents a small biomedical knowledge graph focused on the interaction between drugs and diseases. The design follows the RDFS principles discussed in class by separating general knowledge from instance-level facts. In this sense, the model is not only intended to store individual facts such as *Ibuprofen relieves Arthritis*, but also to define the semantic structure that allows those facts to be interpreted and generalized.

At the schema level, the main classes are **drug** and **disease**. These two classes represent the two central domain concepts of the problem. To capture more specific biomedical categories, **antibiotics** and **antiInflammatory** are modeled as subclasses of **drug**, while **bacterialInfection** and **inflammatoryDisease** are modeled as subclasses of **disease**. This decision allows the graph to express taxonomic knowledge. For example, if **AMOXICILLIN** is asserted as an instance of **antibiotics**, an RDFS reasoner can infer that it is also an instance of **drug**. Similarly, if **ARTHRITIS** is asserted as an instance of **inflammatoryDisease**, it can be inferred that it is also an instance of **disease**.

The relationships between drugs and diseases are represented as RDF properties. The most general property in the model is **hasEffectOn**, which captures the general idea that a drug has an effect on a disease. Its domain is **drug** and its range is **disease**. Therefore, whenever a triple of the form **xhasEffectOny** exists, the subject **x** can be interpreted as a drug and the object **y** as a disease.

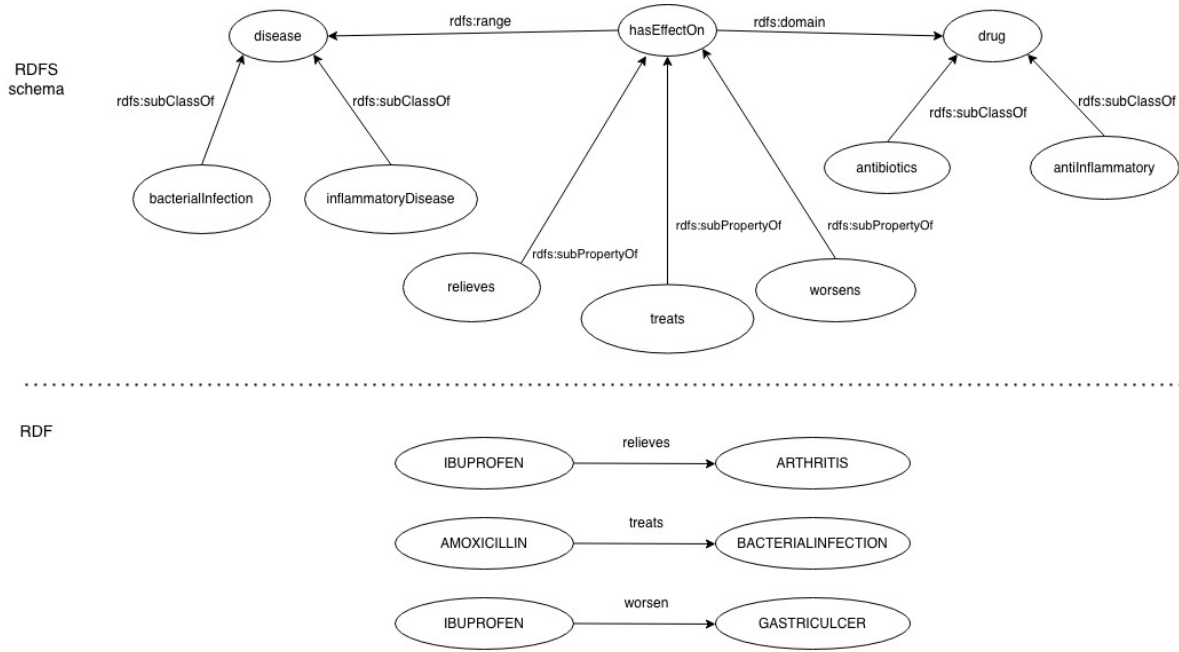


Figure 1: Tentative Graph Design

For readability purposes, the RDF layer only includes a small subset of representative facts from the complete dataset. The full graph generated programmatically contains a significantly larger number of triples and synthetic instances. Including every asserted fact in the graphical representation would reduce clarity and obscure the distinction between schema-level knowledge and instance-level data. Therefore, the diagram focuses on illustrative examples that demonstrate the structure of the model and the intended inference behaviour.

The properties `treats`, `relieves`, and `worsens` are modeled as subproperties of the more general property `hasEffectOn`. This allows the graph to represent different kinds of biomedical interactions while still preserving a common semantic relationship between drugs and diseases. These elements are modeled using `rdfs:subPropertyOf` because they represent relationships between resources rather than categories or types of entities. For example, `relieves` does not describe what `IBUPROFEN` is, but rather how `IBUPROFEN` is related to `ARTHRITIS`. Therefore, `rdfs:subPropertyOf` is the correct RDFS construct instead of `rdfs:subClassOf`.

This property hierarchy enables inference over relationships. If the graph explicitly states that `IBUPROFENrelievesARTHRITIS`, then the reasoner can infer that `IBUPROFENhasEffectOnARTHRITIS`. The same applies to `treats` and `worsen`. This means that a query for all drug–disease pairs connected by the general property `hasEffectOn` can retrieve pairs linked through more specific properties, without manually listing every possible interaction type.

At the facts layer, the model includes the required instances from the assignment. `IBUPROFEN` is typed as an `antiInflammatory`, `AMOXICILLIN` as an `antibiotics`, `ARTHRITIS` as an `inflammatoryDisease`, `BACTERIALINFECTION` as a `bacterialInfection`, and `GASTRICULCER` as a disease-related instance. The factual relationships state that `IBUPROFENrelievesARTHRITIS`, `AMOXICILLIntreatsBACTERIALINFECTION`, and `IBUPROFENworsenGAS`

TRICULCER. These facts are deliberately minimal because several more general facts can be derived from the RDFS schema.

Compared with the Neo4j property graph solution, the RDFS model covers several aspects that are not explicitly represented in the Neo4j version. In Neo4j, labels such as `Drug`, `AntiInflammatory`, or `Disease` are attached directly to nodes, but the graph does not formally state that `AntiInflammatory` is a subclass of `Drug`. Similarly, Neo4j stores relationships such as `RELIEVES`, `TREATS`, and `WORSENS`, but it does not state that they are all special cases of a more general property such as `hasEffectOn`. As a consequence, Neo4j can assert the individual facts, but it cannot infer the higher-level facts unless the query explicitly accounts for all labels and relationship types.

The main advantage of the RDFS solution is therefore its explicit semantics. The model distinguishes between classes, subclasses, properties, subproperties, domains, ranges, and facts. This makes the graph more reusable and easier to extend. New types of drugs or diseases can be added as subclasses, and new kinds of biomedical interactions can be added as subproperties of `hasEffectOn`. In addition, enabling RDFS reasoning in GraphDB allows implicit knowledge to be materialized or queried directly, improving query expressiveness and reducing redundancy in the asserted data.

## B Knowledge Graphs in GraphDB

For simplicity, and because the lab only requires approximately 100 data samples, the whole dataset was generated synthetically. However, it was important to ensure that the examples described in Section A were explicitly represented in the graph. Therefore, the entities and relationships specified in the assignment, such as `IBUPROFEN`, `AMOXICILLIN`, and their interactions with diseases, were manually added to the dataset.

In addition to the subclasses required by the assignment, several extra subclasses of drugs and diseases were introduced in order to enrich the schema and increase the diversity of the generated data. Furthermore, a number of meaningful biomedical examples were manually inserted to verify the correctness of the implementation and to allow the SPARQL queries to return interpretable results.

Finally, 100 additional synthetic drug–disease relationships were generated programmatically. These automatically generated instances ensured that the knowledge graph contained a sufficient dataset for querying and reasoning in GraphDB.

The complete script used to generate the Turtle file is included in Appendix A.

## C Querying the Knowledge Graph in GraphDB

### C.1 All Queries

---

Listing 1: SPARQL: List all drugs

---

```

1 PREFIX ex: <http://example.org/medical-kg/>
2
3 SELECT ?drug
4 WHERE {
5     ?drug a ex:drug .
6 }

```

Listing 2: SPARQL: List all diseases

```

1 PREFIX ex: <http://example.org/medical-kg/>
2
3 SELECT ?disease
4 WHERE {
5     ?disease a ex:disease .
6 }

```

Listing 3: SPARQL: All pairs of drugs and diseases

```

1 PREFIX ex: <http://example.org/medical-kg/>
2
3 SELECT ?drug ?disease
4 WHERE {
5     ?drug ex:hasEffectOn ?disease .
6 }

```

Listing 4: Neo4j: List all drugs

```

1 MATCH (n:Drug)
2 RETURN n;

```

Listing 5: Neo4j: List all diseases

```

1 MATCH (n:Disease)
2 RETURN n;

```

Listing 6: Neo4j: All pairs of drugs and diseases

```

1 MATCH (drug:Drug)-[:TREATS|RELIEVES|WORSENS]->(disease:Disease)
2 RETURN DISTINCT drug.name AS drug,
3                 disease.name AS disease
4 ORDER BY drug, disease;

```

## C.2 Comparison of Neo4j and SPARQL Queries

### C.2.1 Query: List all Drugs

The SPARQL query fetches all resources that are instances of `ex:drug`. Since GraphDB uses RDFS reasoning, the query returns not only explicitly asserted drugs, but also instances belonging to subclasses such as `antibiotics` and `antiInflammatory`. This is possible because the schema defines subclass relationships through `rdfs:subClassOf`. Therefore, instances of subclasses are automatically inferred to also belong to the superclass `drug`.

The equivalent Neo4j query retrieves all nodes labeled `Drug`. However, Neo4j labels do not support subclass reasoning. Consequently, Neo4j only returns nodes explicitly labeled as `Drug`.

### C.2.2 Neo4j Explicit Labels vs GraphDB Inference

To further highlight the semantic differences between GraphDB and Neo4j, two iterations of the Neo4j dataset were tested. In the first iteration, nodes were only assigned their most specific labels, such as **AntiInflammatory** or **Antibiotics**. No explicit **Drug** label was added.

For example, the node representing **IBUPROFEN** was created only with the label **AntiInflammatory**. Similarly, **AMOXICILLIN** was created only with the label **Antibiotics**. In this scenario, the following Neo4j query returned no results:

Listing 7: Neo4j First Iteration: Querying Drug nodes

```
1 MATCH (n:Drug)
2 RETURN n;
```

This behaviour differs significantly from GraphDB. In GraphDB, the SPARQL query:

Listing 8: GraphDB SPARQL Query

```
1 PREFIX ex: <http://example.org/medical-kg/>
2
3 SELECT ?drug
4 WHERE {
5     ?drug a ex:drug .
6 }
```

successfully retrieves resources such as **IBUPROFEN** because GraphDB infers superclass membership through the `rdfs:subClassOf` hierarchy.

In Neo4j, however, no such inference occurs automatically. Therefore, retrieving all drug entities in the first iteration required explicitly enumerating every subclass label:

Listing 9: Neo4j First Iteration: Manual enumeration of labels

```
1 MATCH (n)
2 WHERE n:Antibiotics
3      OR n:AntiInflammatory
4 RETURN n;
```

Although this approach retrieves the expected nodes, it becomes difficult to maintain as the number of subclasses increases.

In the second iteration of the Neo4j dataset, explicit superclass labels were manually added to all subclass instances. For example, nodes labeled **AntiInflammatory** were also labeled as **Drug**. After this modification, the query:

Listing 10: Neo4j Second Iteration: Querying Drug nodes

```
1 MATCH (n:Drug)
2 RETURN n;
```

successfully returned all drug nodes. However, unlike GraphDB, this behaviour was achieved through manual duplication of labels rather than semantic inference.

This experiment illustrates one of the main conceptual differences between property graphs and knowledge graphs. GraphDB relies on explicit semantic definitions and reasoning rules

to infer higher-level knowledge automatically, while Neo4j requires superclass relationships to be manually encoded directly in the data model or queries.

### C.2.3 Query: List all Diseases

The SPARQL query retrieves all data typed as `ex:disease`. Similarly as reasoned in Subsection C.2.1, RDFS reasoning results in GraphDB also returning instances of subclasses such as `bacterialInfection` and `inflammatoryDisease`.

Also as in Subsection C.2.1, the Neo4j query only retrieves nodes directly labeled as `Disease`. In Neo4j, subclass semantics must be handled manually either in the data model or directly in the Cypher query.

### C.2.4 Query: All Pairs of Drugs and Diseases

The SPARQL query gets all pairs connected through the general property `hasEffectOn`. Even though many relationships are explicitly stored using more specific properties such as `treats`, `relieves`, or `worsen`, GraphDB infers that these are all instances of the more general property because they are connected through `rdfs:subPropertyOf`.

The equivalent Neo4j query must explicitly enumerate all relationship types using the syntax `:TREATS|RELIEVES|WORSENS`. Unlike GraphDB, Neo4j cannot infer that these relationships belong to a more general category. Therefore, the Cypher query becomes less reusable and harder to maintain if new relationship types are added in the future.

## C.3 Inferences Involved and Triple Patterns

### C.3.1 Query: List all Drugs

The query pattern for listing all drugs in SPARQL matches both explicit and inferred triples. For example, the graph explicitly contains:

```
1 ex:IBUPROFEN rdf:type ex:antiInflammatory .
2 ex:antiInflammatory rdfs:subClassOf ex:drug .
```

From these triples, GraphDB infers:

```
1 ex:IBUPROFEN rdf:type ex:drug .
```

This inference is generated through the `rdfs:subClassOf` rule. The behaviour was expected because RDFS reasoning propagates instance membership from subclasses to superclasses.

### C.3.2 Query: List all Diseases

The query for listing all diseases also retrieves inferred triples. For example:

```
1 ex:ARTHRITIS rdf:type ex:inflammatoryDisease .
2 ex:inflammatoryDisease rdfs:subClassOf ex:disease .
```

allow GraphDB to infer:

```
1 ex:ARTHRITIS rdf:type ex:disease .
```

This behaviour was expected because RDFS reasoning allows superclass types to be inferred automatically from subclass assertions, similarly to the reasoning in Subsection C.3.1.

### C.3.3 Query: All Pairs of Drugs and Diseases

The final query also retrieves both explicit and inferred relationships. For example, the graph explicitly contains:

```
1 ex:relieves rdfs:subPropertyOf ex:hasEffectOn .
2 ex:IBUPROFEN ex:relieves ex:ARTHRITIS .
```

From these triples, GraphDB infers:

```
1 ex:IBUPROFEN ex:hasEffectOn ex:ARTHRITIS .
```

This behaviour was expected because RDFS reasoning propagates relationships through the hierarchy defined with `rdfs:subPropertyOf`. Therefore, relationships asserted with more specific properties such as `relieves`, `treats`, or `worsen` are automatically inferred as instances of the more general property `hasEffectOn`. The advantage of this approach is that a single SPARQL query can retrieve all drug–disease interactions without explicitly listing every relationship type as needed in the Neo4j query.

## D Neo4j Data Modeling Iterations

To demonstrate the differences between semantic reasoning in GraphDB and explicit labeling in Neo4j, two versions of the Neo4j dataset were evaluated.

In the first iteration, only the most specific labels were assigned to nodes. For example, IBUPROFEN was labeled as `AntiInflammatory` and AMOXICILLIN as `Antibiotics`. No general `Drug` label was assigned. Similarly, diseases only received their most specific labels. Under this modeling approach, queries using the general label `Drug` returned no results because Neo4j does not infer superclass membership automatically.

In the second iteration, additional labels such as `Drug` and `Disease` were manually added to all subclass instances. This allowed generic queries such as `MATCH(n:Drug)` to retrieve all drug nodes successfully. However, unlike GraphDB, this required explicit duplication of superclass information in the graph structure.

The complete Cypher scripts for both iterations are included in Appendix B and Appendix C.

## Appendix A Python RDFLib Implementation

Listing 11: Python program generating the RDF/RDFS biomedical knowledge graph

```

1 from rdflib import Graph, Namespace, RDF, RDFS, Literal
2
3 EX = Namespace("http://example.org/medical-kg/")
4
5 g = Graph()
6 g.bind("ex", EX)
7 g.bind("rdf", RDF)
8 g.bind("rdfs", RDFS)
9
10
11 # Classes from schema
12 drug = EX.drug
13 disease = EX.disease
14 antibiotics = EX.antibiotics
15 antiInflammatory = EX.antiInflammatory
16 bacterialInfection = EX.bacterialInfection
17 inflammatoryDisease = EX.inflammatoryDisease
18
19 for cls in [
20     drug,
21     disease,
22     antibiotics,
23     antiInflammatory,
24     bacterialInfection,
25     inflammatoryDisease,
26 ]:
27     g.add((cls, RDF.type, RDFS.Class))
28
29 g.add((antibiotics, RDFS.subClassOf, drug))
30 g.add((antiInflammatory, RDFS.subClassOf, drug))
31
32 g.add((bacterialInfection, RDFS.subClassOf, disease))
33 g.add((inflammatoryDisease, RDFS.subClassOf, disease))
34
35
36
37 # Extra subclasses
38 extra_drug_subclasses = [
39     "antiviral",
40     "antifungal",
41     "analgesic",
42     "antihypertensive",
43     "antidiabetic",
44     "anticoagulant",
45     "antidepressant",
46     "bronchodilator",
47 ]
48
49 extra_disease_subclasses = [
50     "viralInfection",
51     "fungalInfection",
52     "painDisorder",
53     "cardiovascularDisease",
54     "metabolicDisease",

```



```

55     "respiratoryDisease",
56     "mentalHealthDisorder",
57     "gastrointestinalDisease",
58 ]
59
60 for cls_name in extra_drug_subclasses:
61     cls = EX[cls_name]
62     g.add((cls, RDF.type, RDFS.Class))
63     g.add((cls, RDFS.subClassOf, drug))
64
65 for cls_name in extra_disease_subclasses:
66     cls = EX[cls_name]
67     g.add((cls, RDF.type, RDFS.Class))
68     g.add((cls, RDFS.subClassOf, disease))
69
70 drug_subclasses = [
71     "antibiotics",
72     "antiInflammatory",
73     *extra_drug_subclasses,
74 ]
75
76 disease_subclasses = [
77     "bacterialInfection",
78     "inflammatoryDisease",
79     *extra_disease_subclasses,
80 ]
81
82 # Properties
83 hasEffectOn = EX.hasEffectOn
84 treats = EX.treats
85 relieves = EX.relieves
86 worsens = EX.worsens
87
88 for prop in [hasEffectOn, treats, relieves, worsens]:
89     g.add((prop, RDF.type, RDF.Property))
90
91 # General relationship: drug affects disease
92 g.add((hasEffectOn, RDFS.domain, drug))
93 g.add((hasEffectOn, RDFS.range, disease))
94
95 # Specific effects
96 for prop in [treats, relieves, worsens]:
97     g.add((prop, RDFS.subPropertyOf, hasEffectOn))
98     g.add((prop, RDFS.domain, drug))
99     g.add((prop, RDFS.range, disease))
100
101 # Required instances and facts from the lab
102 required_instances = [
103     (EX.IBUPROFEN, antiInflammatory, "IBUPROFEN"),
104     (EX.AMOXICILLIN, antibiotics, "AMOXICILLIN"),
105     (EX.ARTHRITIS, inflammatoryDisease, "ARTHRITIS"),
106     (EX.BACTERIALINFECTION, bacterialInfection, "BACTERIAL_INFECTION"),
107     (EX.GASTRICULCER, EX.gastrointestinalDisease, "GASTRIC_ULCER"),
108 ]
109
110 for instance, cls, label in required_instances:
111     g.add((instance, RDF.type, cls))
112     g.add((instance, RDFS.label, Literal(label)))

```

```

113
114 g.add((EX.IBUPROFEN, relieves, EX.ARTHRITIS))
115 g.add((EX.AMOXICILLIN, treats, EX.BACTERIALINFECTION))
116 g.add((EX.IBUPROFEN, worsens, EX.GASTRICULCER))
117
118 # Additional instances and relationships to enrich the knowledge graph
119 named_drugs = [
120     ("AZITHROMYCIN", "antibiotics"),
121     ("CIPROFLOXACIN", "antibiotics"),
122     ("DOXYCYCLINE", "antibiotics"),
123     ("NAPROXEN", "antiInflammatory"),
124     ("ASPIRIN", "antiInflammatory"),
125     ("PREDNISONE", "antiInflammatory"),
126     ("ACYCLOVIR", "antiviral"),
127     ("OSELTAMIVIR", "antiviral"),
128     ("FLUCONAZOLE", "antifungal"),
129     ("TERBINAFINE", "antifungal"),
130     ("PARACETAMOL", "analgesic"),
131     ("MORPHINE", "analgesic"),
132     ("LISINOPRIL", "antihypertensive"),
133     ("AMLODIPINE", "antihypertensive"),
134     ("METFORMIN", "antidiabetic"),
135     ("INSULIN", "antidiabetic"),
136     ("WARFARIN", "anticoagulant"),
137     ("HEPARIN", "anticoagulant"),
138     ("SERTRALINE", "antidepressant"),
139     ("FLUOXETINE", "antidepressant"),
140     ("SALBUTAMOL", "bronchodilator"),
141     ("FORMOTEROL", "bronchodilator"),
142 ]
143
144 named_diseases = [
145     ("PNEUMONIA", "bacterialInfection"),
146     ("TUBERCULOSIS", "bacterialInfection"),
147     ("STREPTTHROAT", "bacterialInfection"),
148     ("CROHNSDISEASE", "inflammatoryDisease"),
149     ("PSORIASIS", "inflammatoryDisease"),
150     ("LUPUS", "inflammatoryDisease"),
151     ("INFLUENZA", "viralInfection"),
152     ("HERPES", "viralInfection"),
153     ("COVID19", "viralInfection"),
154     ("ATHLETESFOOT", "fungalInfection"),
155     ("CANDIDIASIS", "fungalInfection"),
156     ("MIGRAINE", "painDisorder"),
157     ("BACKPAIN", "painDisorder"),
158     ("HYPERTENSION", "cardiovascularDisease"),
159     ("HEARTFAILURE", "cardiovascularDisease"),
160     ("TYPE2DIABETES", "metabolicDisease"),
161     ("TYPE1DIABETES", "metabolicDisease"),
162     ("ASTHMA", "respiratoryDisease"),
163     ("COPD", "respiratoryDisease"),
164     ("DEPRESSION", "mentalHealthDisorder"),
165     ("ANXIETY", "mentalHealthDisorder"),
166     ("GERD", "gastrointestinalDisease"),
167 ]
168
169 for name, cls_name in named_drugs:
170     g.add((EX[name], RDF.type, EX[cls_name]))

```

```

171     g.add((EX[name], RDFS.label, Literal(name)))
172
173 for name, cls_name in named_diseases:
174     g.add((EX[name], RDF.type, EX[cls_name]))
175     g.add((EX[name], RDFS.label, Literal(name)))
176
177 relationships = [
178     ("AMOXICILLIN", "treats", "PNEUMONIA"),
179     ("DOXYCYCLINE", "treats", "TUBERCULOSIS"),
180     ("AZITHROMYCIN", "treats", "STREPTHROAT"),
181     ("CIPROFLOXACIN", "treats", "BACTERIALINFECTION"),
182     ("IBUPROFEN", "relieves", "ARTHRITIS"),
183     ("NAPROXEN", "relieves", "ARTHRITIS"),
184     ("PREDNISONE", "relieves", "CROHNSDISEASE"),
185     ("ASPIRIN", "relieves", "BACKPAIN"),
186     ("ACYCLOVIR", "treats", "HERPES"),
187     ("OSELTAMIVIR", "treats", "INFLUENZA"),
188     ("FLUCONAZOLE", "treats", "CANDIDIASIS"),
189     ("TERBINAFINE", "treats", "ATHLETESFOOT"),
190     ("PARACETAMOL", "relieves", "MIGRAINE"),
191     ("MORPHINE", "relieves", "BACKPAIN"),
192     ("LISINOPRIL", "treats", "HYPERTENSION"),
193     ("AMLODIPINE", "treats", "HEARTFAILURE"),
194     ("METFORMIN", "treats", "TYPE2DIABETES"),
195     ("INSULIN", "treats", "TYPE1DIABETES"),
196     ("SALBUTAMOL", "relieves", "ASTHMA"),
197     ("FORMOTEROL", "relieves", "COPD"),
198     ("SERTRALINE", "treats", "DEPRESSION"),
199     ("FLUOXETINE", "treats", "ANXIETY"),
200     ("IBUPROFEN", "worsens", "GASTRICULCER"),
201     ("ASPIRIN", "worsens", "GERD"),
202     ("NAPROXEN", "worsens", "GASTRICULCER"),
203 ]
204
205 for drug_name, relation_name, disease_name in relationships:
206     g.add((EX[drug_name], EX[relation_name], EX[disease_name]))
207
208 # Add 100 relationships between the named drugs and diseases
209 for i in range(1, 101):
210     drug_instance = EX[f"DRUGINSTANCE{i}"]
211     disease_instance = EX[f"DISEASEINSTANCE{i}"]
212
213     drug_class = EX[drug_subclasses[i % len(drug_subclasses)]]
214     disease_class = EX[disease_subclasses[i % len(disease_subclasses)]]
215
216     g.add((drug_instance, RDF.type, drug_class))
217     g.add((drug_instance, RDFS.label, Literal(f"DRUG_INSTANCE_{i}")))
218
219     g.add((disease_instance, RDF.type, disease_class))
220     g.add((disease_instance, RDFS.label, Literal(f"DISEASE_INSTANCE_{i}")))
221
222     if i % 3 == 0:
223         g.add((drug_instance, treats, disease_instance))
224     elif i % 3 == 1:
225         g.add((drug_instance, relieves, disease_instance))
226     else:
227         g.add((drug_instance, worsens, disease_instance))

```

```

228
229 # Serialize the graph to a Turtle file
230 output_file = "medical_kg.ttl"
231 g.serialize(destination=output_file, format="turtle")
232
233 print(f"Created_{output_file}")
234 print(f"Total_explicit_triples:{len(g)}")

```

## Appendix B Neo4j First Iteration

The first Neo4j iteration only assigned the most specific labels to nodes. No explicit Drug or Disease labels were added.

Listing 12: Neo4j First Iteration

```

1 MATCH (n)
2 DETACH DELETE n;
3
4 CREATE
5   (ibuprofen:AntiInflammatory {name: "IBUPROFEN"}),
6   (amoxicillin:Antibiotics {name: "AMOXICILLIN"}),
7
8   (arthritis:InflammatoryDisease {name: "ARTHRITIS"}),
9   (bacterialInfection:BacterialInfection {name: "BACTERIALINFECTION"}),
10  (gastricUlcer:GastrointestinalDisease {name: "GASTRICULCER"}),
11
12  (ibuprofen)-[:RELIEVES]->(arthritis),
13  (amoxicillin)-[:TREATS]->(bacterialInfection),
14  (ibuprofen)-[:WORSEN]->(gastricUlcer);

```

## Appendix C Neo4j Second Iteration

The second Neo4j iteration manually added the superclass labels Drug and Disease.

Listing 13: Neo4j Second Iteration

```

1 MATCH (n)
2 WHERE n:Antibiotics
3      OR n:AntiInflammatory
4 SET n:Drug;
5
6 MATCH (n)
7 WHERE n:BacterialInfection
8      OR n:InflammatoryDisease
9      OR n:GastrointestinalDisease
10 SET n:Disease;

```